INSIDE STLC ACADEMY

PRACTICAL TEST AUTOMATION WITH PLAYWRIGHT

CI/CD Integration Guide (GitHub Actions)

Run your Playwright tests automatically on every push.

Introduction

Automated tests deliver the most value when they run automatically. This guide shows you how to run Playwright in GitHub Actions so every push and pull request is checked.

Instructions

- Use the workflow generated by npm init playwright (or add one).
- Install browsers in CI with npx playwright install --with-deps.
- Upload the HTML report and traces as artefacts.

Worked Example (workflow steps)

1. Checkout the repo (actions/checkout)
2. Setup Node (actions/setup-node)
3. npm ci
4. npx playwright install --with-deps
5. npx playwright test
6. Upload playwright-report/ as an artefact (always)

Reliability in CI

Setting	Why
retries: 2 in CI	Absorb rare infra blips (not real flakiness)
trace: on-first-retry	Diagnose failures from artefacts
workers tuned	Balance speed vs runner resources

Practical Exercise

Push your Playwright project to GitHub with a workflow, open a pull request, and confirm the tests run and the report uploads as an artefact.

Best Practices

- Run on push and pull_request.
- Always upload the report/traces (if: always()).
- Keep the pipeline fast and reliable.

Common Mistakes

- Forgetting --with-deps, so browsers fail to launch.
- Not uploading artefacts, so CI failures are undiagnosable.
- Letting flaky tests erode trust in the pipeline.

> **INSIDE STLC PRO TIP**
>
> A green CI badge on a public repo is a powerful portfolio signal. It shows you can not only write tests, but wire them into a real automated pipeline — exactly what employers want.